Descripción Ver entrega

Límite de entrega: jueves, 14 de marzo de 2024, 12:40
Ficheros requeridos: cuentabancaria.py, prueba.py (Descargar)
Tipo de trabajo: Individual

Gestión de Cuentas Bancarias

Desarrolle en el módulo *cuentabancaria* una **clase en Python** de nombre `CuentaBancaria` que modele una **cuenta bancaria básica**, atendiendo a las siguientes características y requerimientos.

Clase CuentaBancaria

- **Consideraciones generales**
  - **Añada los atributos de instancia y de clase** que considere necesarios para cumplir los requisitos solicitados.
  - La clase debe construirse **ocultando** adecuadamente los **atributos** que se usen.
  - No se requieren más comprobaciones que las indicadas.

- **Inicializador (constructor)**: Recibe en una string el nombre del titular de la cuenta a crear.

- **Propiedades de instancia**:
  - `codigo_cuenta` (solo lectura): string, código de la cuenta; inmutable, con el formato "`CB-`*`nnn`*", donde *`nnn`* son tres cifras numéricas que se deberán ir asignando a las sucesivas cuentas conforme vayan creándose, empezando por 001 y siguiendo con 002, 003, etc. (se recomienda usar el formato `:03d` en una f-string). *Si no se resolviera la generación automática, utilícese el código de cuenta "CB-001".*
  - `titular` (solo lectura): string, nombre del titular de la cuenta.
  - `saldo` (solo lectura): flotante, saldo actual de la cuenta (inicialmente, 0.0).

- **Métodos de instancia**:
  - `depositar(cantidad)`: Añade la cantidad al saldo, salvo si la cantidad fuera negativa, en cuyo caso se deberá lanzar una excepción `ValueError`.
  - `retirar(cantidad)`: Resta la cantidad al saldo, salvo si el saldo fuera insuficiente o la cantidad fuera negativa, en cuyo caso se deberá lanzar una excepción `ValueError`.

- **Métodos de clase**:
  - `obtener_total_cuentas()`: Devuelve el número total de cuentas creadas.
  - `total_saldo_titular(titular)`: Devuelve la suma de los saldos de todas las cuentas del titular.

- **Representación informal**:
  - Contendrá la información de la cuenta, siguiendo el formato:
    `<código_cuenta> - Titular: <titular>, Saldo: <saldo>`
    Por ejemplo, para el siguiente código se debe mostrar lo indicado:

```
cuenta1 = CuentaBancaria('José Rodríguez')
cuenta1.depositar(12)
print(cuenta1)  # CB-001 - Titular: José Rodríguez, Saldo: 12.0
cuenta2 = CuentaBancaria('Juan García')
cuenta2.depositar(33)
print(cuenta2)  # CB-002 - Titular: Juan García, Saldo: 33.0
```

Pruebas

Se dispone de un fichero `prueba.py` modificable para probar su clase al ejecutarlo.

Notas: Se recomienda desplegar la pestaña Descripción justo antes de pulsar Evaluar, y desplegar también la pestaña Comentarios au ? cuando se muestre algún mensaje en la de Compilación.

English version

Bank Account Management 🏦

Develop a class in Python called **CuentaBancaria** within the module **cuentabancaria**, modeling a basic bank account while adhering to the following characteristics and requirements.

CuentaBancaria Class 💳

General Considerations

- Add the necessary **instance and class attributes and methods** to meet the specified requirements.
- The class must be designed **hiding attributes properly**.
- No additional validations are required beyond those explicitly stated.

Initializer (Constructor)

- Receives a **string** with the name of the account holder.

Instance Properties

- **codigo_cuenta** *(read-only)*: A **string** representing the account code. Its value must be generated upon object creation following the format `"CB-XXX"`, where **XXX** is a three-digit sequential number starting at **001** (then 002, 003, etc.). It is recommended to use the format `":03d"` in an **f-string**.
  - If automatic generation is not implemented, use the default account code `"CB-001"`.
- **titular** *(read-only)*: A **string** with the name of the account holder.
- **saldo** *(read-only)*: A **float** representing the current account balance (**initially 0.0**).

Instance Methods

- **depositar(cantidad)**:
  - Adds **cantidad** to the balance.
  - If **cantidad** is negative, a **ValueError exception** must be raised.
- **retirar(cantidad)**:
  - Subtracts **cantidad** from the balance.
  - If the balance is **insufficient** or the **amount is negative**, a **ValueError exception** must be raised.

Class Methods

- **obtener_total_cuentas()**:
  - Returns the **total number of created accounts**.
- **total_saldo_titular(titular)**:
  - Returns the **sum of balances** from all accounts belonging to the same **titular**.

Informal Representation

- The string representation of the account must follow the format:

```
<codigo_cuenta> - Titular: <titular>, Saldo: <saldo>
```

- Example output for the following code:

```
cuenta1 = CuentaBancaria('José Rodríguez')
cuenta1.depositar(12)
print(cuenta1)  # CB-001 - Titular: José Rodríguez, Saldo: 12.0

cuenta2 = CuentaBancaria('Juan García')
cuenta2.depositar(33)
print(cuenta2)  # CB-002 - Titular: Juan García, Saldo: 33.0
```

Testing

- A file named **prueba.py** is available where you can write code to **test your class**.

©2012-2015 - Laurent Pointal Mémento v2.0.6
License Creative Commons Attribution 4

Python 3 Cheat Sheet

Latest version on :
https://perso.limsi.fr/pointal/python:memento

Base Types

integer, float, boolean, string, bytes

```
int 783  0 -192  0b010 0o642 0xF3
float 9.23 0.0  -1.7e-6
bool True False
str "One\nTwo"
    'I\'m'
bytes b"toto\xfe\775"
```

zero *binary* *octal* *hexa*

escaped new line *escaped '* *escaped tab* *hexadecimal* *octal* *$\times 10^{-6}$*

Multiline string: `"""X\tY\tZ 1\t2\t3"""`

immutables

Container Types

- **ordered sequences**, fast index access, repeatable values

```
list [1,5,9]   ["x",11,8.9]   ["mot"]   []
tuple (1,5,9)  11,"y",7.4     ("mot",)  ()
str bytes
```

Non modifiable values (immutables) *expression with only comas → `tuple`* *(ordered sequences of chars / bytes)*

- **key containers**, no *a priori* order, fast key access, each key is unique

dictionary `dict {"key":"value"}` `dict(a=3,b=4,k="v")` `{}`
(key/value associations) `{1:"one",3:"three",2:"two",3.14:"π"}`

collection `set {"key1","key2"}` `{1,9,3,0}` `set()`

keys=hashable values (base types, immutables…) `frozenset` *immutable set* *empty*

Identifiers

for variables, functions, modules, classes… names

`a…zA…Z_` followed by `a…zA…Z_0…9`
- diacritics allowed but should be avoided
- language keywords forbidden
- lower/UPPER case discrimination

☺ `a toto x7 y_max BigOne`
☹ `~~By and for~~`

Variables assignment

`=`

assignment ⇔ **binding** of a *name* with a *value*
1) *evaluation of right side expression value*
2) *assignment in order with left side names*

```
x=1.2+8+sin(y)
a=b=c=0      assignment to same value
y,z,r=9.2,-7.6,0   multiple assignments
a,b=b,a      values swap
a,*b=seq     unpacking of sequence in
*a,b=seq     item and list
x+=3         increment ⇔ x=x+3
x-=2         decrement ⇔ x=x-2
x=None       « undefined » constant value
del x        remove name x
```

and `*=` `/=` `%=` …

Conversions

`type(expression)`

```
int("15") → 15
int("3f",16) → 63          can specify integer number base in 2nd parameter
int(15.56) → 15            truncate decimal part
float("-11.24e8") → -1124000000.0
round(15.56,1)→15.6        rounding to 1 decimal (0 decimal → integer number)
bool(x)  False for null x, empty container x , None or False x ; True for other x
str(x)→ "…"   representation string of x for display (cf. formatting on the back)
chr(64)→'@'  ord('@')→64   code ↔ char
repr(x)→ "…"  literal representation string of x
bytes([72,9,64]) → b'H\t@'
list("abc") → ['a','b','c']
dict([(3,"three"),(1,"one")]) → {1:'one',3:'three'}
set(["one","two"]) → {'one','two'}
separator str and sequence of str → assembled str
  ':'.join(['toto','12','pswd']) → 'toto:12:pswd'
str splitted on whitespaces → list of str
  "words with  spaces".split() → ['words','with','spaces']
str splitted on separator str → list of str
  "1,4,8,2".split(",") → ['1','4','8','2']
sequence of one type → list of another type (via list comprehension)
  [int(x) for x in ('1','29','-3')] → [1,29,-3]
```

Sequence Containers Indexing

for lists, tuples, strings, bytes…

negative index	−5	−4	−3	−2	−1
positive index	0	1	2	3	4
`lst=[10,`	`20,`	`30,`	`40,`	`50]`	
positive slice	0	1	2	3	4	5
negative slice	−5	−4	−3	−2	−1	

Items count
`len(lst)→5`

index from 0
(here from 0 to 4)

Individual access to **items** via `lst[index]`
`lst[0]→10` ⇒ *first one* `lst[1]→20`
`lst[-1]→50` ⇒ *last one* `lst[-2]→40`

On mutable sequences (`list`), remove with `del lst[3]` and modify with assignment `lst[4]=25`

Access to **sub-sequences** via `lst[start slice:end slice:step]`

```
lst[:-1]→[10,20,30,40]   lst[::-1]→[50,40,30,20,10]   lst[1:3]→[20,30]   lst[:3]→[10,20,30]
lst[1:-1]→[20,30,40]     lst[::-2]→[50,30,10]         lst[-3:-1]→[30,40] lst[3:]→[40,50]
lst[::2]→[10,30,50]      lst[:]→[10,20,30,40,50] shallow copy of sequence
```

Missing slice indication → from start / up to end.
On mutable sequences (`list`), remove with `del lst[3:5]` and modify with assignment `lst[1:4]=[15,25]`

Boolean Logic

Comparisons : `< > <= >= == !=` (*boolean results*) ≤ ≥ = ≠

`a and b` logical and *both simultaneously*

`a or b` logical or *one or other or both*

pitfall : `and` *and* `or` *return value of* `a` *or of* `b` *(under shortcut evaluation).*
⇒ *ensure that* `a` *and* `b` *are booleans.*

`not a` logical not

`True` `False` True and False constants

Statements Blocks

parent statement :
statement block 1…
⋮
parent statement :
statement block2…
⋮
next statement after block 1

indentation

configure editor to insert 4 spaces in place of an indentation tab.

Modules/Names Imports

module `truc`⇔file `truc.py`

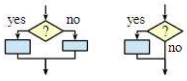
```
from monmod import nom1,nom2 as fct
  →direct access to names, renaming with as
import monmod →access via monmod.nom1 …
```

modules and packages searched in *python path* (cf `sys.path`)

Conditional Statement

statement block executed only if a condition is true

```
if logical condition :
    statements block
```



Can go with several *elif*, *elif*… and only one final *else*. Only the block of first true condition is executed.

```
if age<=18:
  state="Kid"
elif age>65:
  state="Retired"
else:
  state="Active"
```

with a var `x`:
`if bool(x)==True:` ⇔ `if x:`
`if bool(x)==False:` ⇔ `if not x:`

Maths

floating numbers… approximated values *angles in radians*

Operators: `+ - * / // % **`
Priority `(…)` × ÷ ↑ ↑ a^b integer ÷ ÷ remainder

`@` → matrix × *python3.5+`numpy`*

```
(1+5.3)*2→12.6
abs(-3.2)→3.2
round(3.57,1)→3.6
pow(4,3)→64.0
```

usual order of operations

```
from math import sin,pi…
sin(pi/4)→0.707…
cos(2*pi/3)→-0.4999…
sqrt(81)→9.0  √
log(e**2)→2.0
ceil(12.5)→13
floor(12.5)→12
```

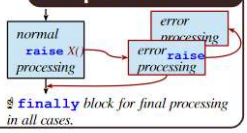
modules `math`, `statistics`, `random`, `decimal`, `fractions`, `numpy`, *etc. (cf. doc)*

Exceptions on Errors

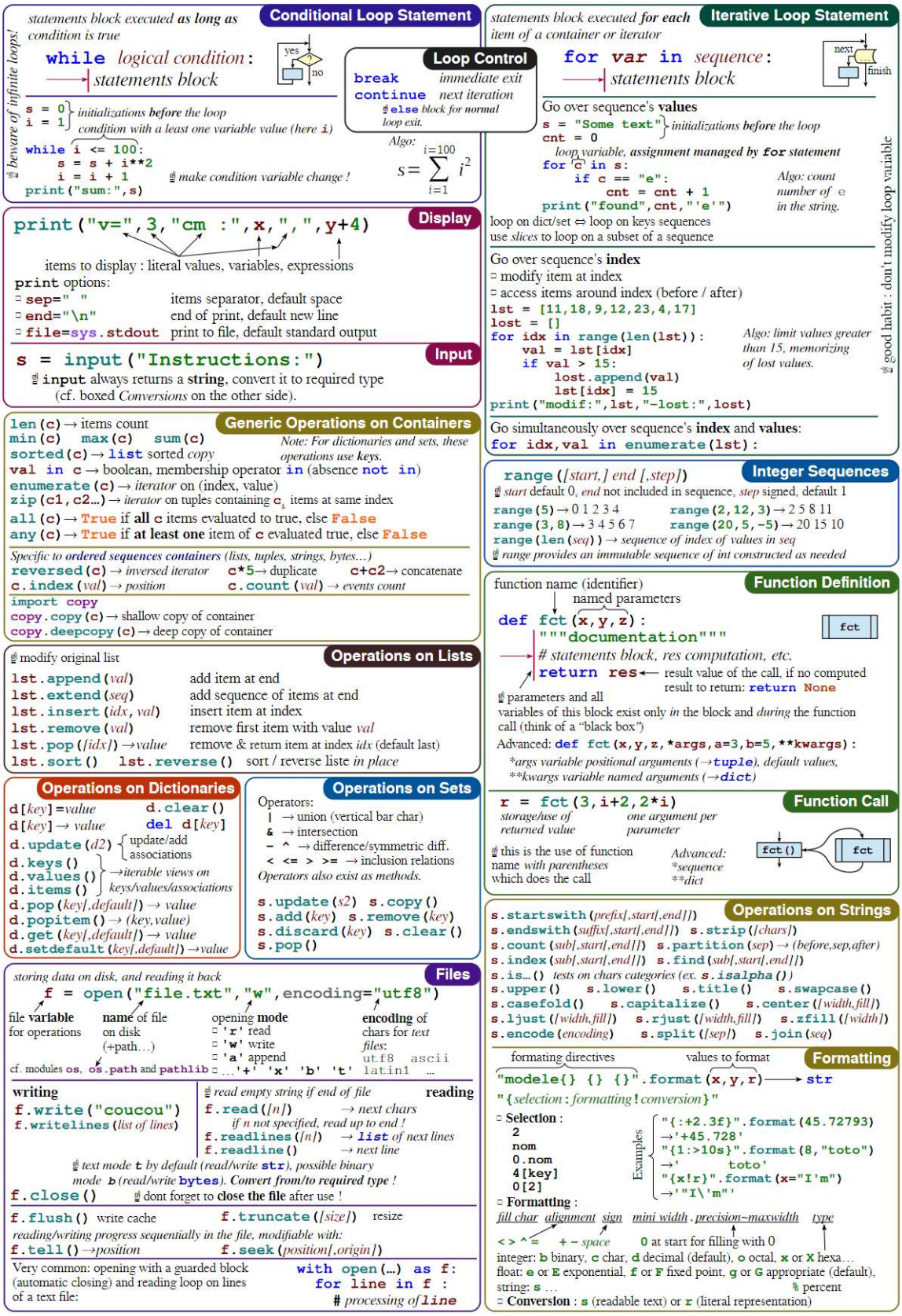
Signaling an error:
`raise` *ExcClass(…)*

Errors processing:

```
try:
    normal procesising block
except Exception as e:
    error processing block
```



`finally` *block for final processing in all cases.*



Ficheros requeridos

cuentabancaria.py

```
# ESCRIBA AQUÍ LA CLASE
class CuentaBancaria:
    # descomente la siguiente línea si quiere usarla
    #   titular suma: dict[str, float] = {}  # diccionario titular: suma saldos
```

prueba.py

```python
from cuentabancaria import CuentaBancaria
# Ejemplo de uso
cuenta1 = CuentaBancaria('José Rodríguez')
cuenta1.depositar(12)
print(cuenta1)  # CB-001 - Titular: José Rodríguez, Saldo: 12
# Intento de deposito negativo
try:
    cuenta1.depositar(-1)
except ValueError as e:
    print(f"{e.__class__.__name__}: {e}")

# Intento de retirada superior al saldo
try:
    cuenta1.retirar(100)
except ValueError as e:
    print(f"{e.__class__.__name__}: {e}")

cuenta2 = CuentaBancaria('Juan García')
cuenta2.depositar(33)
print(cuenta2)  # CB-002 - Titular: Juan García, Saldo: 33

cuenta3 = CuentaBancaria('José Rodríguez')
cuenta3.depositar(50)
print(cuenta3)  # CB-003 - Titular: José Rodríguez, Saldo: 50

# Prueba de métodos de clase
print("Cuentas abiertas", CuentaBancaria.obtener_total_cuentas())  # 3
print("Total saldo José Rodríguez",
      CuentaBancaria.total_saldo_titular('José Rodríguez'))  # 62

```